

Week 1 Study Sheet

C for Embedded, STM32F401 Peripherals & Measurement Habits — EE0827

How to Use This Sheet

This is **not** a copy of the slides. It contains worked examples, decision guides, and answered questions that go beyond the lecture. Use it alongside your slides for review.

Worked Examples — Pointers

Example 1: Trace the output

```
uint32_t arr[] = {0xAA, 0xBB, 0xCC, 0xDD};
uint32_t *p = &arr[1];
printf("%X\n", *p);           // Line A
printf("%X\n", *(p + 2));    // Line B
printf("%X\n", p[-1]);       // Line C
```

Line	p points to	Expression	Value
A	arr[1]	*p	0xBB
B	arr[1]	*(p+2) = arr[3]	0xDD
C	arr[1]	p[-1] = *(p-1) = arr[0]	0xAA

Key insight: Negative indexing on pointers is valid C. `p[-1]` accesses the element *before* where `p` points.

Example 2: sizeof trap

```
void send_buffer(uint8_t *buf) {
    int len = sizeof(buf);    // WRONG: always 4 on 32-bit MCU
}

uint8_t data[128];
int len = sizeof(data);      // CORRECT: 128
send_buffer(data);           // Inside function, sizeof gives 4
```

Rule: When an array is passed to a function, it decays to a pointer. You lose size information. Always pass length as a separate argument.

Example 3: Struct padding — predict the size

```
struct A { uint8_t x; uint32_t y; uint8_t z; }; // sizeof = ?  
struct B { uint32_t y; uint8_t x; uint8_t z; }; // sizeof = ?
```

Answer:

- struct A: $1 + (3 \text{ pad}) + 4 + 1 + (3 \text{ pad}) = \mathbf{12 \text{ bytes}}$
- struct B: $4 + 1 + 1 + (2 \text{ pad}) = \mathbf{8 \text{ bytes}}$

Rule: ARM Cortex-M requires 4-byte values aligned to 4-byte boundaries. Compiler inserts invisible padding. Order fields from largest to smallest.

Worked Examples — Volatile and ISR

Example 4: What goes wrong without volatile?

```
uint32_t tick = 0;           // Missing volatile!

void TIM2_IRQHandler(void) { tick++; }

int main(void) {
    HAL_TIM_Base_Start_IT(&htim2);
    while (tick < 1000) { }    // May loop forever
    printf("Done!\n");
}
```

What happens at -O2: The compiler sees that `tick` is never modified *in the visible code path* of `main()`. It loads `tick` into a register once and keeps checking the register — never re-reading from RAM. The ISR updates RAM, but `main` never sees it.

Fix: `volatile uint32_t tick = 0;` forces every read to go to RAM.

Mental model: `volatile` means “someone else can change this behind the compiler’s back.”

Example 5: Why not log inside the ISR?

```
void TIM2_IRQHandler(void) {
    HAL_GPIO_WritePin(TRACE_Port, TRACE_Pin, GPIO_PIN_SET);
    printf("tick=%lu\n", tick);    // BAD: takes 500+ us at 115200 baud
    HAL_GPIO_WritePin(TRACE_Port, TRACE_Pin, GPIO_PIN_RESET);
}
```

At 115200 baud, transmitting `"tick=12345\r\n"` (12 chars) takes:

$$t = \frac{12 \times 10 \text{ bits}}{115200} \approx 1.04 \text{ ms}$$

Your ISR period is 1 ms. The ISR takes *longer than the period* — interrupts pile up, UART corrupts, system hangs.

Rule: ISR budget is 1–5% of the period. For 1 kHz: keep under 10 μ s.

Worked Examples — STM32 Software Stack

Example 6: HAL vs register — same operation, two ways

Task: Set PA5 HIGH (turn on LED).

```
// HAL approach - portable, readable
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);

// Register approach - fast, chip-specific
GPIOA->BSRR = (1 << 5); // Atomic set via Bit Set/Reset Register
```

What HAL does underneath: `HAL_GPIO_WritePin()` checks the pin state argument, then writes to either the set or reset half of BSRR. The register version skips the check and writes directly — one instruction vs several.

When to use which: HAL for application code, registers for time-critical ISRs where every cycle matters.

Example 7: Reading a HAL handle

```
extern UART_HandleTypeDef huart2; // Generated by CubeMX

// What can you learn from the handle?
printf("Baud: %lu\n", huart2.Init.BaudRate); // 115200
printf("Instance: %p\n", huart2.Instance); // 0x40004400 (USART2)

// The Instance pointer gives you direct register access:
huart2.Instance->SR; // Status register - same as USART2->SR
```

Key insight: Every HAL handle contains an `Instance` pointer to the CMSIS register struct. HAL and CMSIS are not separate worlds — HAL wraps CMSIS.

Example 8: Callback routing — why check Instance?

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        tick_count++;
    } else if (htim->Instance == TIM3) {
        // Different timer, different action
    }
}
```

Why this matters: All timers share the same callback function. Without the `Instance` check, TIM3 events would incorrectly increment `tick_count`. Always guard with `htim->Instance == TIMx`.

Worked Examples — Timer Calculations

Example 9: Design a timer for a different frequency

Problem: You need a 50 Hz interrupt (20 ms period) from the 84 MHz APB1 clock. Find PSC and ARR.

Method: $f_{update} = \frac{f_{clock}}{(PSC+1)(ARR+1)}$, so $(PSC + 1)(ARR + 1) = \frac{84\,000\,000}{50} = 1\,680\,000$

Several valid solutions:

PSC (register)	ARR (register)	Check
8399	199	$8400 \times 200 = 1\,680\,000$
839	1999	$840 \times 2000 = 1\,680\,000$
41999	39	$42000 \times 40 = 1\,680\,000$

Choosing between them: Smaller ARR = coarser duty cycle resolution if using PWM later. Prefer PSC=8399, ARR=199 for general use.

Decision Guides

“Which software layer should I use?”

Situation	Layer	Why
Normal application code	HAL	Readable, CubeMX generates init code, callbacks built-in
Performance-critical ISR	LL or register	Minimal overhead, predictable timing
Understanding what HAL does	CMSIS registers	Maps directly to datasheet
Feature not exposed by HAL	Register	Direct access to all hardware
Mixing in one project	HAL + LL	HAL for init, LL for fast runtime paths

Default: Start with HAL. Drop down only when you have a measured reason.

“Do I need volatile?”

Ask: *Can this variable change without the current function explicitly writing to it?*

Scenario	volatile?	Why
Local loop counter	No	Only this code modifies it
Global incremented in ISR, read in main	Yes	ISR modifies behind compiler's back
Hardware register (GPIOA->IDR)	Yes	Hardware changes it
Local variable passed to HAL function	No	Compiler can see the function call
Flag set by DMA complete callback	Yes	DMA/callback modifies asynchronously

“Which GPIO mode?”

You need to...	Mode	Pull	Speed
Read a button	Input	Pull-up (active low) or pull-down	Low
Drive an LED / trace pin	Output push-pull	None	High (trace) or Low (LED)
Connect to UART TX/RX	Alternate function (AF7)	None	High
Read analog voltage (ADC)	Analog	None	N/A
Share a bus (I2C)	Output open-drain	External pull-up	Medium

“My measurement is wrong — what broke?”

Period is...	Likely cause
10x too slow	Forgot to subtract 1 from PSC or ARR (off-by-one)
10x too fast	Used wrong clock tree value (APB1 vs AHB)
Exactly 2x off	Timer clock prescaler in RCC is not what you assumed
Drifting slowly	Using HSI (internal RC, $\pm 1\%$) — normal
Random / unstable	Noise on probe, bad ground connection, or code bug

Quick-Reference Formulas

$$f_{update} = \frac{f_{clock}}{(PSC + 1)(ARR + 1)}$$

$$\text{ISR budget: } t_{ISR} < 0.05 \times T_{period}$$

$$\text{UART byte time: } t = \frac{10 \text{ bits}}{\text{baudrate}}$$

Answers to Pre-Lab Self-Assessment

1. **ptr++ for uint32_t *ptr:** Advances address by 4 bytes (`sizeof(uint32_t)`), not 1 byte.
2. **Why volatile for tick_count?** Without it, compiler may cache the value in a register and never re-read from RAM. The ISR updates RAM, so `main` would never see changes at `-02`.
3. **htim->Instance:** `htim` is a pointer to `TIM_HandleTypeDef`. The `->` operator dereferences the pointer and accesses the `Instance` member (which itself is a pointer to the timer's register block, e.g. `TIM2`).